Progettazione di Briscola con 4 giocatori

Elaborato da Falzetta Christian, Latini Mattia, Senzacqua Alessandro

Obbiettivo:

Vogliamo realizzare la Briscola a coppie in client–server, l'idea chiave è: il server è "autoritativo" (decide cosa è valido e aggiorna lo stato), mentre i client inviano solo intenzioni/azioni e mostrano lo stato ricevuto. In questo modo si evitano irregolarità, blocco della connessione e ambiguità su prese e punteggi.

Requisiti di gioco:

In modalità a coppie i compagni stanno uno di fronte all'altro, le prese della coppia confluiscono in un unico "mazzetto" e si sommano ai fini del punteggio. Le regole di presa dipendono da seme/briscola e dall'ordine di gioco, quindi il server deve poter determinare in modo univoco la carta vincente della mano e chi inizia il turno successivo. La smazzata totalizza 120 punti e tipicamente vince la coppia che arriva al punteggio più alto.

Componenti dell'architettura client-server:

- Client: visione del tavolo, gestione input (selezione carta), animazioni, mantiene una copia dello stato dei punti della partita e invia i comandi.
- Server: creazione "stanze" da 4 giocatori, validazione mosse, applicazione regole (presa, pesca, turni), calcolo punteggi e fine smazzata/partita.
- Classe MessageHeader
- Classe Mazzo.
- Carta: classe con gli attributi: seme, valore, punti.
- Classe giocatore.
- Classe squadra.
- Classe partita.

Regole di gioco:

Il gioco inizia dopo la connessione tra il server e i client (2 client), Il seme della briscola viene generato in modo casuale e il mazziere (server) distribuisce le carte ai 4 giocatori.
Durante il gioco, la presa avviene secondo i seguenti criteri.

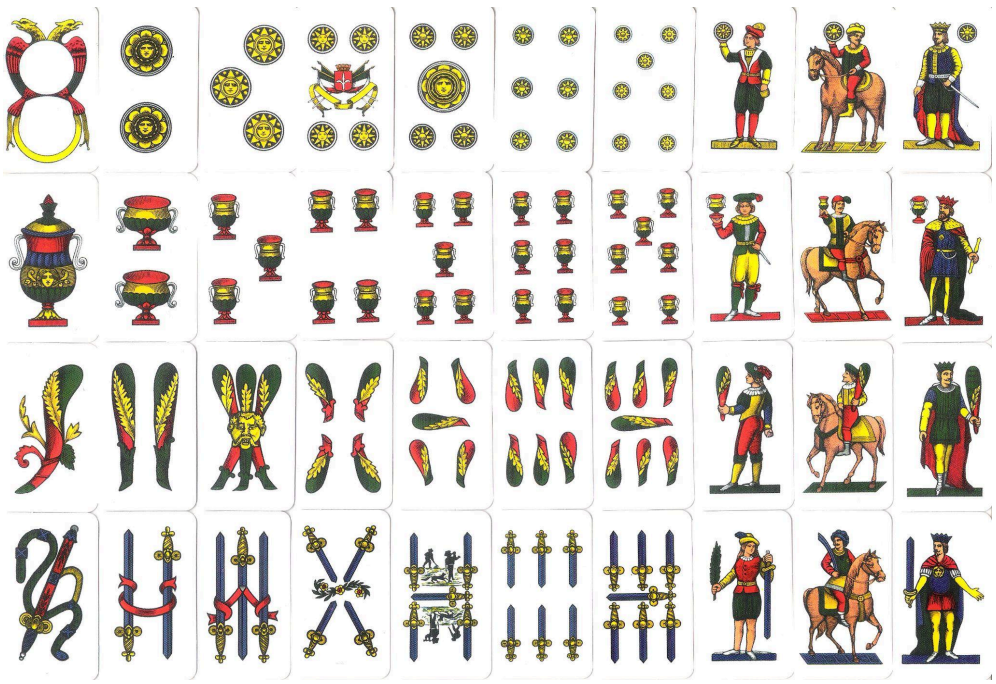
A. Se non viene giocata alcuna carta di briscola e…

- Se il giocatore di mano cala una carta di un certo seme e gli altri giocatori ne calano una di un seme diverso: cattura sempre il primo giocatore, qualsiasi sia il valore della carta giocata dall'avversario.
- Se il primo di mano cala una carta di un certo seme e gli altri giocatori successivi calano una carta del medesimo seme: cattura il giocatore che ha calato la carta di maggior valore.

B. Se si gioca almeno una briscola…
- Se se ne gioca solo una, il giocatore che la cala effettua sempre la cattura, qualsiasi sia il valore facciale della sua carta in rapporto a quella avversaria.
- Se più di un giocatore cala una briscola, invece, vince la carta di maggior valore.

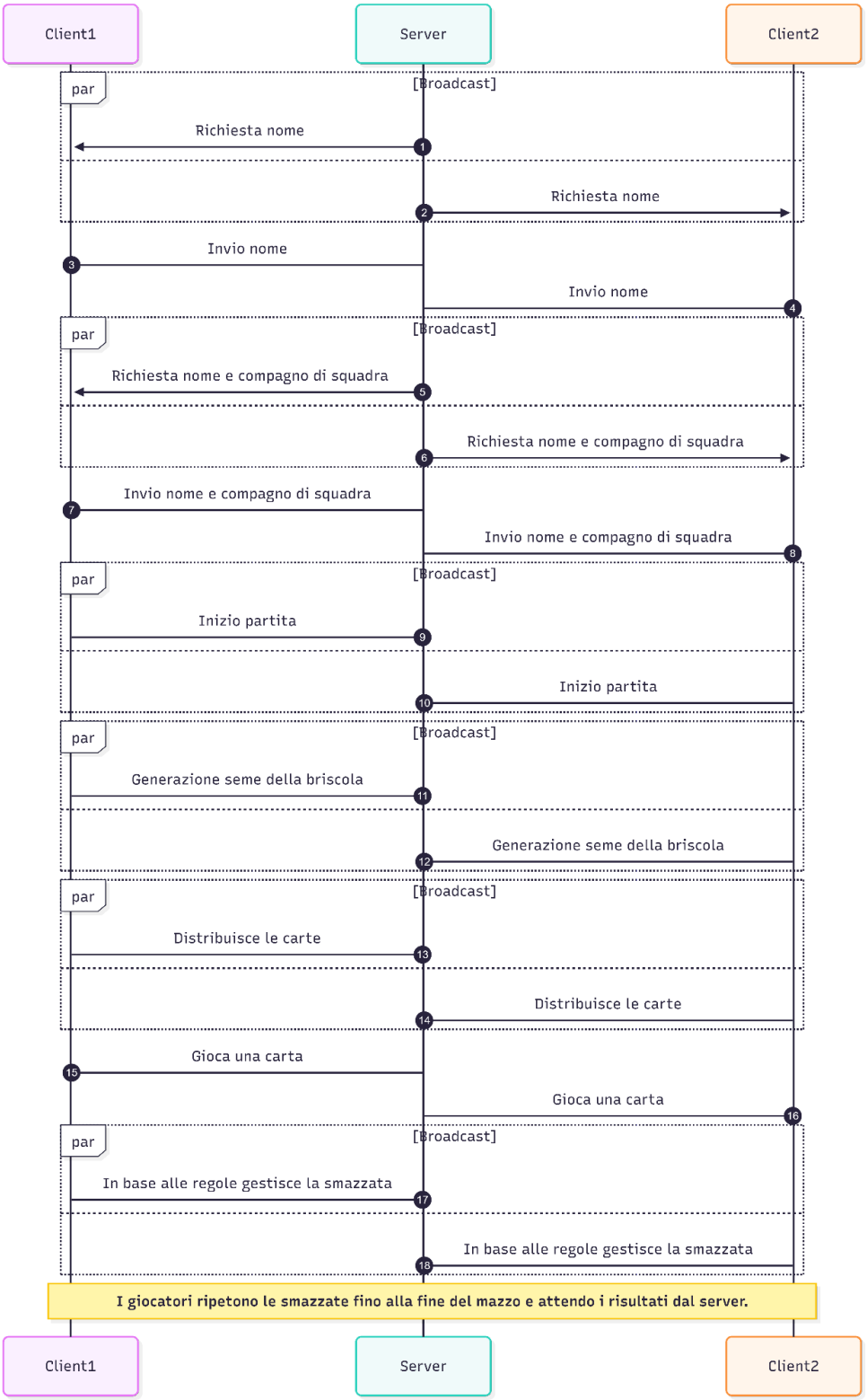
La squadra che effettua la presa raccoglie le due carte e le pone coperte davanti a sé. Le successive catture si accumuleranno a formare un mazzetto sempre coperto. È vietato guardare le proprie o le altrui carte già oggetto di cattura. Dopo la prima presa, il giocatore che l'ha effettuata pesca dal tallone la prima carta disponibile e ricostituisce in mano il proprio ventaglio di tre carte. Anche gli altri giocatori pescano la carta successiva e il gioco prosegue con una nuova calata. A giocare per primo è sempre chi ha effettuato l'ultima presa. Quando il tallone si esaurisce, l'ultima carta (quella scoperta che ha indicato il seme di briscola) spetta al giocatore che ha ottenuto la presa precedente. Da questo momento il gioco prosegue fino all'esaurimento delle carte in mano ai giocatori. Infine, si procede al calcolo dei punti. La somma dei punteggi di ogni seme (illustrata in seguito) è uguale a 30, quindi l'intero mazzo ha un punteggio totale pari a 120: bastano 61 punti per assicurarsi la smazzata. Nel caso in cui in una smazzata entrambe le squadre ottengono 60 punti, va ripetuta con lo stesso mazziere. Altrimenti vince la squadra con il punteggio più alto. Una partita è normalmente costituita da 1 smazzata.

<u>Punteggi:</u>



Ai fini della presa e del calcolo del punteggio a fine smazzata, le carte, di qualsiasi seme siano, hanno il seguente valore: Asso 11 punti; Tre 10 punti; Re 4 punti; Cavallo/Donna 3 punti; Fante 2 punti.
Tutte le altre carte non hanno alcun valore e vengono perciò dette “scartine”.

Diagramma di sequenza:



Definizione del set di messaggi per l'applicazione:

Nel quadro del progetto, vogliamo definire un protocollo proprietario per lo scambio di informazioni tra client e server.
Più precisamente vogliamo definire:

- Le **informazioni** che il client e il server potranno scambiarsi (sulla base delle operazioni selezionate dai client)
- Il **formato** che dovranno avere i messaggi, che dovrà chiaramente essere idoneo al trasporto delle informazioni di cui sopra

Le informazioni relative a ciascun giocatore sono le seguenti:

- Codice giocatore (un codice univoco a 8 cifre)
- Nome
- Nome della squadra

Le informazioni relative a ciascuna squadra sono le seguenti:

- Codice squadra (un codice univoco a 8 cifre)
- Nome del giocatore1
- Nome del giocatore2

Il codice giocatore e squadra è quindi l'informazione che collega le informazioni relative ai giocatori e alla squadra.
Nel seguito ci focalizziamo quindi sulla definizione dei messaggi con i quali client e server si trasmetteranno queste semplici informazioni.
Il protocollo definisce lo scambio di messaggi tra client e server, il server è **autoritativo**:

- Decide se una mossa è valida.
- Aggiorna lo stato del gioco.
- Calcola prese e punteggi.
- Gestisce i turni e la fine della partita.

I client:

- Inviano azioni (intenzioni).
- Ricevono lo stato aggiornato.
- Mostrano il tavolo.

Questo evita irregolarità e ambiguità sulle regole del gioco.

Formato generale dei messaggi dell'applicazione:

Un messaggio del nostro semplice protocollo sarà contenuto in una **stringa** e sarà in generale composto da più sottostringhe che separeremo mediante punto e virgola (;).
Ciascuna sottostringa rappresenta un **campo** del messaggio:
Ad esempio il messaggio
"01;48372916"
Contiene i campi (in questo caso tutti numerici, ma potrebbe contenere anche caratteri alfabetici):
01
48372916
Ciascun campo ha un significato specifico all'interno del messaggio. Ad esempio (come assumeremo nel seguito) il primo campo potrebbe contenere un numero che identifica il tipo di messaggio (**Message Identifier o ID**).
Il tipo di messaggio determinerà il numero e la lunghezza (nonchè il significato) dei campi successivi.

Nel nostro semplice set di messaggi, tutti i messaggi condividono quindi il seguente formato generale:

- Intestazione (2 caratteri): contiene l'ID del messaggio.
- Altri campi, il cui numero e la cui lunghezza saranno dipendenti dal particolare messaggio. Per uno specifico campo, tuttavia, useremo sempre lo stesso numero di byte (ad esempio: 10 byte per il campo che contiene il nome di un giocatore, 10 byte per il campo che contiene il nome della squadra…).

Nota: dal momento che useremo campi di lunghezza predeterminata, l'utilizzo di un separatore (;) tra i vari campi in realtà non sarebbe necessario.
*Tuttavia la presenza del separatore renderà i messaggi molto più leggibili per noi (e, in fase di implementazione, con il metodo **split** della classe **String** sarà semplice separare i campi!).*

Elenco dei messaggi:

Per la nostra semplice applicazione definiamo i seguenti messaggi. Per il momento specifichiamo solo l'ID, poi dettaglieremo il formato completo dei messaggi con i vari campi. In generale, in questo semplice protocollo, sarà sempre il client a mandare dei messaggi di richiesta, ai quali il server farà seguire dei messaggi di risposta.

ID 19: Conferma connessione (da Server a Client)
ID 04: Inizio partita (da Server ai Client)
ID 64: Carte distribuite (da Server a Client)
ID 67: Carta accettata (da Server ai Client)
ID 92: Presa effettuata (da Server ai Client)
ID 34: Nuovo turno (da Server ai Client)
ID 52: Carta pescata (da Server a Client)
ID 12: Fine smazzata (da Server ai Client)
ID 56: Risultati (da Server ai Client)
ID 79: Richiesta connessione (da Client a Server)
ID 09: Giocatore pronto (da Client a Server)
ID 63: Gioca carta (da Client a Server)
ID 10: Esito connessione (da Server a Client)
ID 17: Esito giocatore pronto (da Server a Client)
ID 14: Esito giocata carta (da Server a Client)
ID 31: Esito carta pescata (da Server a Client)
ID 44: Esito inizio partita (da Server ai Client)

Formato dei messaggi:

Vediamo ora quanti e quali potrebbero essere i campi di ciascuno dei messaggi che abbiamo individuato.

Conferma connessione

- ID messaggio (2 caratteri "19")
- Codice giocatore (8 caratteri)
- Codice squadra (8 caratteri)

Questo messaggio non ha bisogno di altri campi.

Inizio partita

- ID messaggio (2 caratteri “04”)
- Valoreseme (16 caratteri)
- Turno (Codice giocatore iniziale, 8 caratteri)

Carte distribuite

- ID messaggio (2 caratteri “64”)
- Valoreseme1 (16 caratteri)
- Valoreseme2 (16 caratteri)
- Valoreseme3 (16 caratteri)

Carta accettata

- ID messaggio (2 caratteri “67”)
- Codice giocatore (8 caratteri)
- Valoreseme1 (16 caratteri)

Presa effettuata

- ID messaggio (2 caratteri “92”)
- Codice giocatore (8 caratteri)
- Valoreseme1 (16 caratteri)
- Punteggio (2 caratteri)

Nuovo turno

- ID messaggio (2 caratteri “34”)
- Codice giocatore (8 caratteri)

Carta pescata

- ID messaggio (2 caratteri “52”)
- Valoreseme1 (16 caratteri)

Fine smazzata

- ID messaggio (2 caratteri “12”)
- Punteggio squadra1 (3 caratteri)
- Punteggio squadra2 (3 caratteri)

Risultati

- ID messaggio (2 caratteri “56”)
- Nome Squadra (8 caratteri)

Richiesta connessione

- ID messaggio (2 caratteri “79”)
- Nome giocatore (16 caratteri)
- Nome squadra (16 caratteri)

Giocatore pronto

- ID messaggio (2 caratteri “09”)
- Codice giocatore (8 caratteri)

Gioca carta

- ID messaggio (2 caratteri “63”)
- Codice giocatore (8 caratteri)
- Valoreseme1 (16 caratteri)
- **Esito connessione**
- ID messaggio (2 caratteri “10”)
- Esito (1 carattere: “0” per OK, “1” per FAIL)

Esito giocatore pronto

- ID messaggio (2 caratteri “17”)

- Esito (1 carattere: “0” per OK, “1” per FAIL)

Esito giocata carta

- ID messaggio (2 caratteri “14”)
- Esito (1 carattere: “0” per OK, “1” per FAIL)

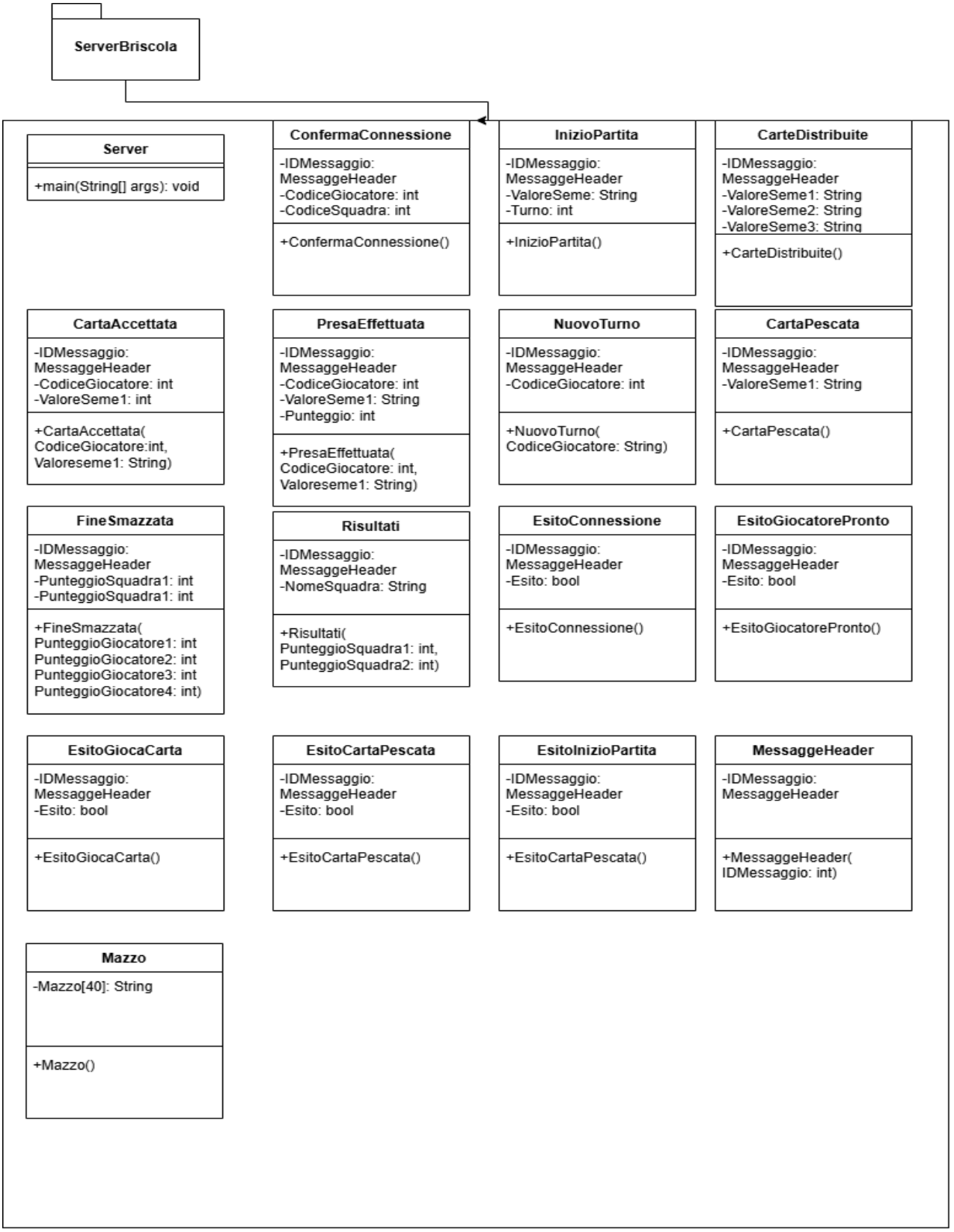
Esito carta pescata

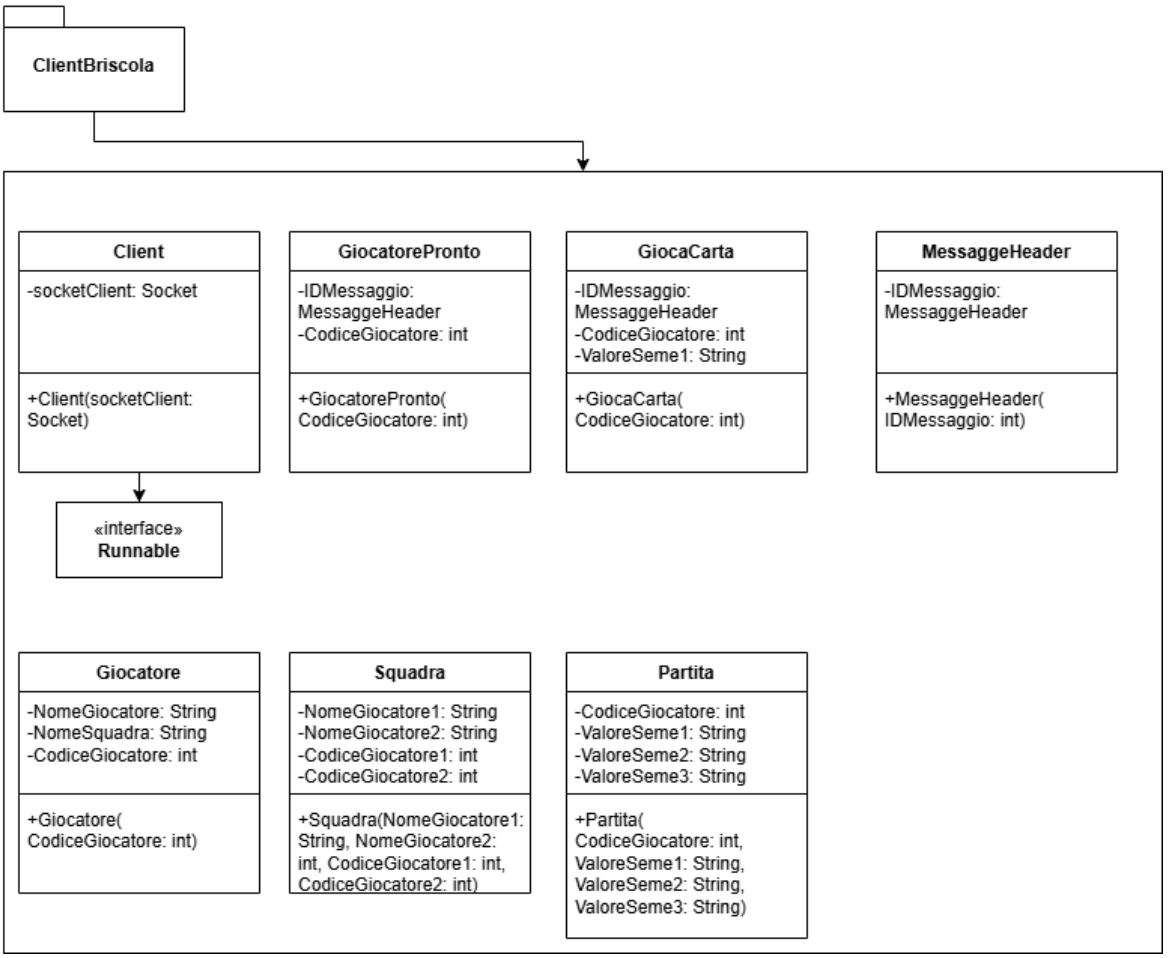
- ID messaggio (2 caratteri “31”)
- Esito (1 carattere: “0” per OK, “1” per FAIL)

Esito inizio partita

- ID messaggio (2 caratteri “44”)
- Esito (1 carattere: “0” per OK, “1” per FAIL)

UML:


ServerBriscola
Server
+main(String[] args): void
ConfermaConnessione
-IDMessaggio: MessaggeHeader
-CodiceGiocatore: int
-CodiceSquadra: int
+ConfermaConnessione()
InizioPartita
-IDMessaggio: MessaggeHeader
-ValoreSeme: String
-Turno: int
+InizioPartita()
CarteDistribuite
-IDMessaggio: MessaggeHeader
-ValoreSeme1: String
-ValoreSeme2: String
-ValoreSeme3: String
+CarteDistribuite()
CartaAccettata
-IDMessaggio: MessaggeHeader
-CodiceGiocatore: int
-ValoreSeme1: int
+CartaAccettata(CodiceGiocatore:int, Valoreseme1: String)
PresaEffettuata
-IDMessaggio: MessaggeHeader
-CodiceGiocatore: int
-ValoreSeme1: String
-Punteggio: int
+PresaEffettuata(CodiceGiocatore: int, Valoreseme1: String)
NuovoTurno
-IDMessaggio: MessaggeHeader
-CodiceGiocatore: int
+NuovoTurno(CodiceGiocatore: String)
CartaPescata
-IDMessaggio: MessaggeHeader
-ValoreSeme1: String
+CartaPescata()
FineSmazzata
-IDMessaggio: MessaggeHeader
-PunteggioSquadra1: int
-PunteggioSquadra1: int
+FineSmazzata(PunteggioGiocatore1: int PunteggioGiocatore2: int PunteggioGiocatore3: int PunteggioGiocatore4: int)
Risultati
-IDMessaggio: MessaggeHeader
-NomeSquadra: String
+Risultati(PunteggioSquadra1: int, PunteggioSquadra2: int)
EsitoConnessione
-IDMessaggio: MessaggeHeader
-Esito: bool
+EsitoConnessione()
EsitoGiocatorePronto
-IDMessaggio: MessaggeHeader
-Esito: bool
+EsitoGiocatorePronto()
EsitoGiocaCarta
-IDMessaggio: MessaggeHeader
-Esito: bool
+EsitoGiocaCarta()
EsitoCartaPescata
-IDMessaggio: MessaggeHeader
-Esito: bool
+EsitoCartaPescata()
EsitoInizioPartita
-IDMessaggio: MessaggeHeader
-Esito: bool
+EsitoCartaPescata()
MessaggeHeader
-IDMessaggio: MessaggeHeader
+MessaggeHeader(IDMessaggio: int)
Mazzo
-Mazzo[40]: String
+Mazzo()


ClientBriscola
Client
-socketClient: Socket
+Client(socketClient: Socket)
«interface»
Runnable
GiocatorePronto
-IDMessaggio: MessaggeHeader
-CodiceGiocatore: int
+GiocatorePronto(CodiceGiocatore: int)
GiocaCarta
-IDMessaggio: MessaggeHeader
-CodiceGiocatore: int
-ValoreSeme1: String
+GiocaCarta(CodiceGiocatore: int)
MessaggeHeader
-IDMessaggio: MessaggeHeader
+MessaggeHeader(IDMessaggio: int)
Giocatore
-NomeGiocatore: String
-NomeSquadra: String
-CodiceGiocatore: int
+Giocatore(CodiceGiocatore: int)
Squadra
-NomeGiocatore1: String
-NomeGiocatore2: String
-CodiceGiocatore1: int
-CodiceGiocatore2: int
+Squadra(NomeGiocatore1: String, NomeGiocatore2: int, CodiceGiocatore1: int, CodiceGiocatore2: int)
Partita
-CodiceGiocatore: int
-ValoreSeme1: String
-ValoreSeme2: String
-ValoreSeme3: String
+Partita(CodiceGiocatore: int, ValoreSeme1: String, ValoreSeme2: String, ValoreSeme3: String)